

Laboratorio di Informatica 1

Corso di Laurea in Fisica

Federico Nati, Uni. Milano-Bicocca, 2021

Lezione 4, 14 marzo 2021

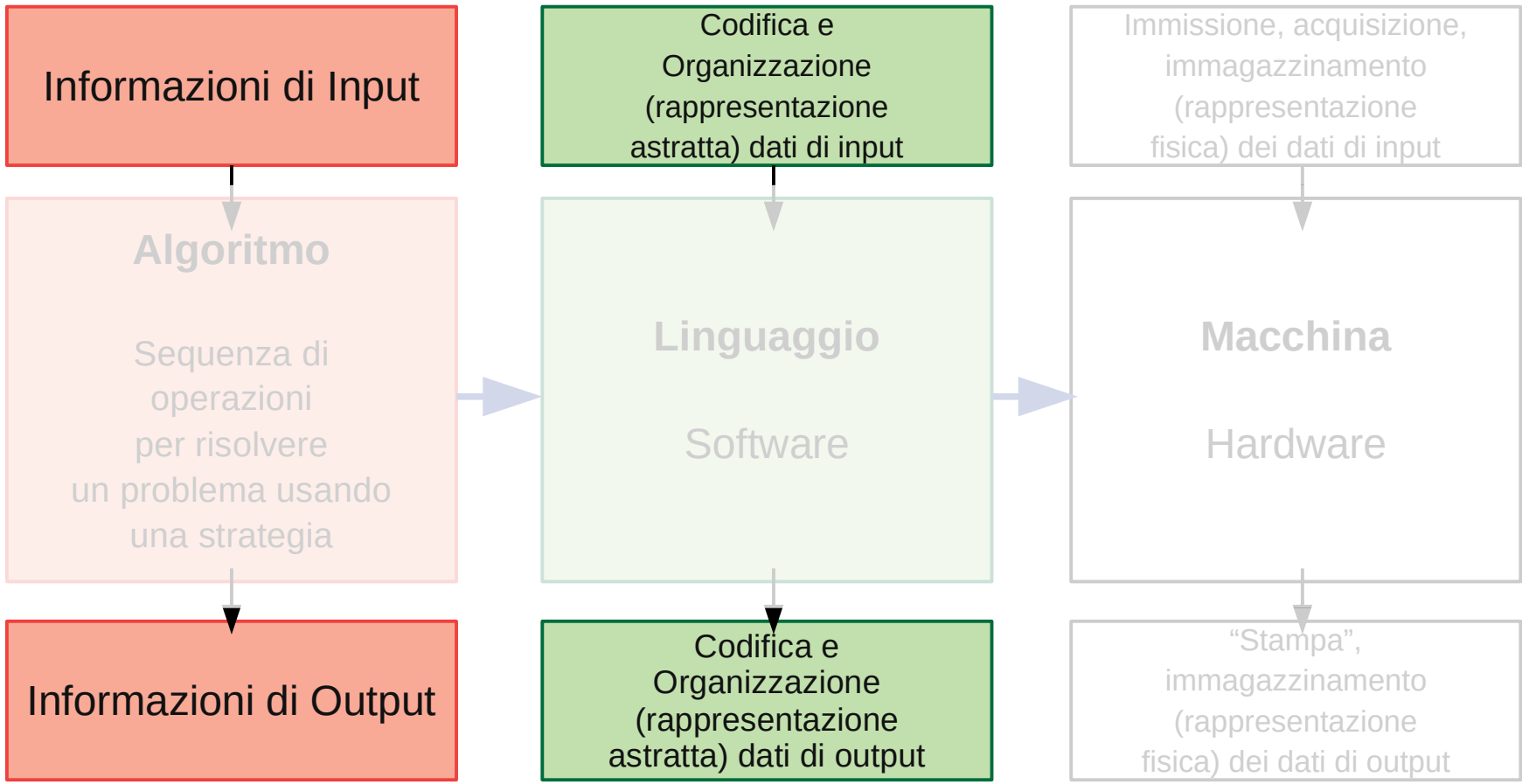
ARGOMENTI

Tra gli argomenti visti nelle scorse lezioni:

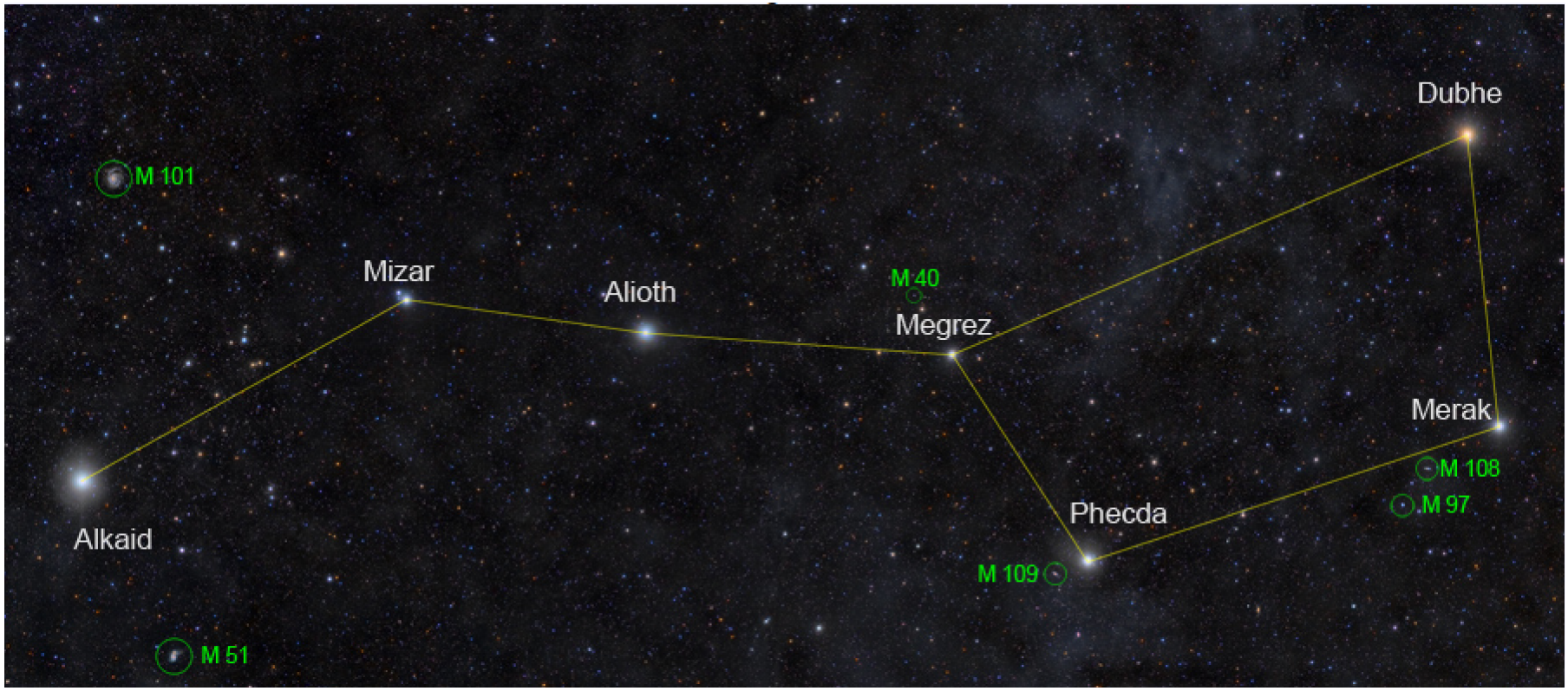
- Rappresentazione di informazioni con codice binario
- Dati e algoritrmi
- Rappresentazione di dati in un calcolatore
- Un primo tipo di dato strutturato: l’array
- Le funzioni
- Campo di visibilità (scope) degli identificatori

Oggi vedremo:

- Tipi di dati strutturati in C: le strutture (o record, o struct)
- Rappresentazione di dati in un calcolatore
- Rappresentazione di numeri interi
- Numeri reali, rappresentazione e problema dell'approssimazione
- Rappresentazione di caratteri


Informazioni di Input
Codifica e Organizzazione (rappresentazione astratta) dati di input
Immissione, acquisizione, immagazzinamento (rappresentazione fisica) dei dati di input
Algoritmo
Sequenza di operazioni per risolvere un problema usando una strategia
Linguaggio
Software
Macchina
Hardware
Informazioni di Output
Codifica e Organizzazione (rappresentazione astratta) dati di output
“Stampa”, immagazzinamento (rappresentazione fisica) dei dati di output




Dubhe
M 101
Mizar
Alioth
M 40
Megrez
Merak
M 108
M 97
Phecda
Alkaid
M 109
M 51

Nome Proprio	Nomenclatura di Bayer	Magnitudine apparente	Distanza (al)
Dubhe	α UMa	1.8	124
Merak	β UMa	2.4	79
Phecda	γ UMa	2.4	84
Megrez	δ UMa	3.3	81
Alioth	ε UMa	1.8	81
Mizar	ζ UMa	2.1	78
Alkaid	η UMa	1.9	101

Stella

Costellazione

Questo è il nome del tipo

```
struct Stella {
    char NomeProprio[64];
    char NomeBayer[64];
    double Magnitudine;
    int Distanza;
};

struct Stella UrsaMaior[7];

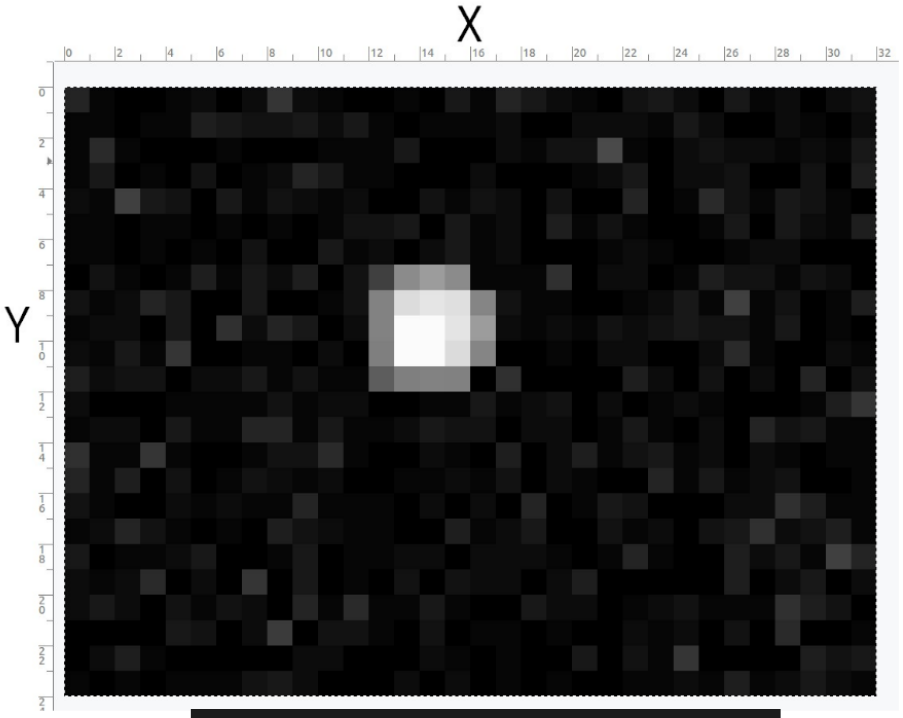
UrsaMaior[0].Distanza = 124;
```

	Nome Proprio	Nomenclatura di Bayer	Magnitudine apparente	Distanza (al)
0	Dubhe	α UMa	1.8	124
1	Merak	β UMa	2.4	79
2	Phecda	γ UMa	2.4	84
3	Megrez	δ UMa	3.3	81
4	Alioth	ε UMa	1.8	81
5	Mizar	ζ UMa	2.1	78
6	Alkaid	η UMa	1.9	101

Notiamo che proprio come nell'esempio del foglio di carta a quadretti, dobbia decidere da subito “quanti quadretti”, cioè quanta memoria occuperà la struttura. Come per ogni tipo di dato, il C deve sapere quale è la sua dimensione.



```
struct TimeDate {
    int ora;
    int minuti;
    int secondi;
    int giorno;
    int mese;
    int anno;
};
```




```
struct Pixel {
    int x;
    int y;
    int v;
};
```

Altri esempi:

- I dati anagrafici di una persona
- Gli studenti di una classe
- Numeri complessi
- Aggregazione di dati statistici
- Eccetera,...

DATI STRUTTURATI

- In generale gli algoritmi lavorano con dati strutturati

- Tali strutture di dati possono contenere informazioni eterogenee, ciascuna identificata da un campo, ognuno con un suo nome e tipo

- Nota terminologica:
  - strutture, struct, record sono sinonimi che indicano aggregati di dati;
  - ciascun singolo membro di una struttura (il nome della colonna se la pensiamo come una tabella) è detto campo (field);
  - ciascun dato strutturato (ciascuna riga se pensiamo alla tabella) è detto record

- È possibile definire vettori di record (la tabella nell'esempio precedente)

- Un campo può essere a sua volta un record, o un vettore, o altro

- In questo modo è possibile comporre strutture via via più complesse (per esempio una struttura di dati relativi a persone contiene dei campi “data” che sono a loro volta strutture di dati)

DEFINIZIONE DI UNA STRUTTURA

Nome della struttura

```
struct punto {
    float x;
    float y;
    float z;
};
```

Campi della struttura

Al contrario delle funzioni, qui dobbiamo terminare con ";"

Dove posizionare la definizione?

Fuori da ogni funzione e prima di dove viene utilizzata

Tipi dei campi

Ogni campo in una struttura ha un suo tipo. I tipi possono essere diversi tra di loro e possono anche essere altre strutture o array

Perché?

Non stiamo definendo codice che viene eseguito, ma un modo di raggruppare dati (un "tipo di dato")

DEFINIZIONE DI UNA STRUTTURA

È anche possibile dichiarare delle variabili o degli array del tipo "`struct nome_struttura`" direttamente dopo la definizione della struttura stessa.

```
struct coordtemp {
  int x, y; /* Coordinate */
  float temp; /* Temperatura */
} sp, pt[NP];
```

ESEMPI DI UTILIZZO DI STRUTTURE

```
float x = 4.5;
float y = 3.4;
float z = 5.6;
```



Tipo della variabile Nome della variabile

```
struct punto p;
p.x = 4.5;
p.y = 3.4;
p.z = 5.6;
```

Campi della struttura

x, y e z rappresentano
le coordinate di un punto
in uno spazio a tre dimensioni.

STRUTTURE IN MEMORIA

```
struct punto {
  float x;
  float y;
  float z;
};
```

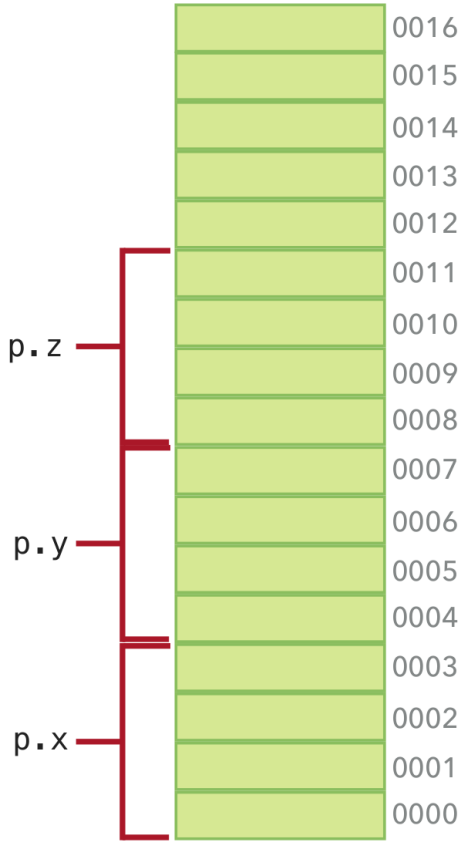
Campi

Tipi dei campi

Tipo di dato

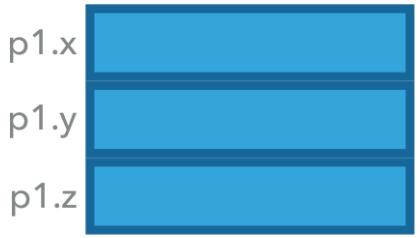
```
struct punto p;
```

Nome della variabile



ASSEGNAMENTO E COPIA CON LE STRUTTURE

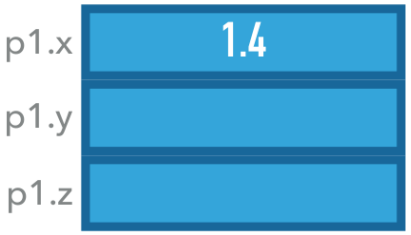
```
struct punto {
  float x;
  float y;
  float z;
};
```




```
struct punto p1;
p1.x = 1.4;
p1.y = 3.2;
p1.z = 5;
struct punto p2;
p2 = p1;
p1.x = 6;
p2.y = 7;
```

ASSEGNAMENTO E COPIA CON LE STRUTTURE

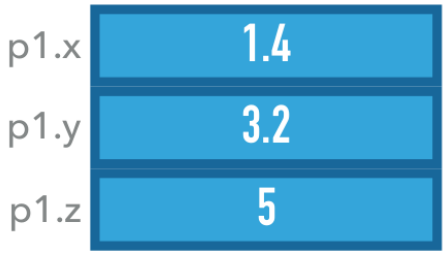
```
struct punto {
  float x;
  float y;
  float z;
};
```




```
struct punto p1;
p1.x = 1.4;
p1.y = 3.2;
p1.z = 5;
struct punto p2;
p2 = p1;
p1.x = 6;
p2.y = 7;
```

ASSEGNAMENTO E COPIA CON LE STRUTTURE

```
struct punto {
  float x;
  float y;
  float z;
};
```

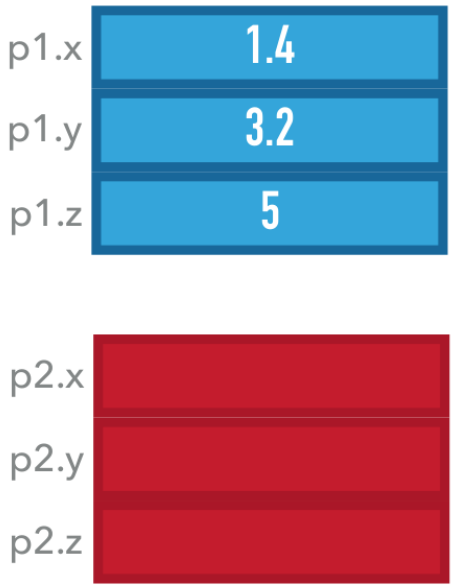



```
struct punto p1;
p1.x = 1.4;
p1.y = 3.2;
p1.z = 5;
struct punto p2;
p2 = p1;
p1.x = 6;
p2.y = 7;
```

ASSEGNAMENTO E COPIA CON LE STRUTTURE

```
struct punto {
  float x;
  float y;
  float z;
};
```

```
struct punto p1;
p1.x = 1.4;
p1.y = 3.2;
p1.z = 5;
struct punto p2;
p2 = p1;
p1.x = 6;
p2.y = 7;
```

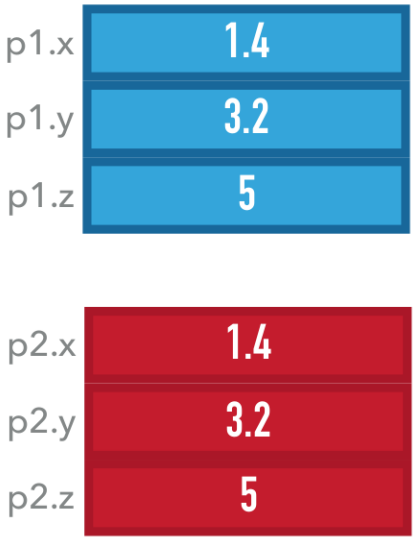


ASSEGNAMENTO E COPIA CON LE STRUTTURE

```
struct punto {
  float x;
  float y;
  float z;
};



struct punto p1;
p1.x = 1.4;
p1.y = 3.2;
p1.z = 5;
struct punto p2;
p2 = p1;
p1.x = 6;
p2.y = 7;
```

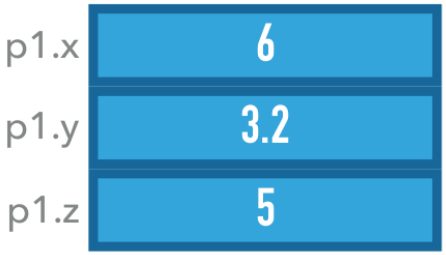


ASSEGNAMENTO E COPIA CON LE STRUTTURE

```
struct punto {
  float x;
  float y;
  float z;
};



struct punto p1;
p1.x = 1.4;
p1.y = 3.2;
p1.z = 5;
struct punto p2;
p2 = p1;
p1.x = 6;
p2.y = 7;
```

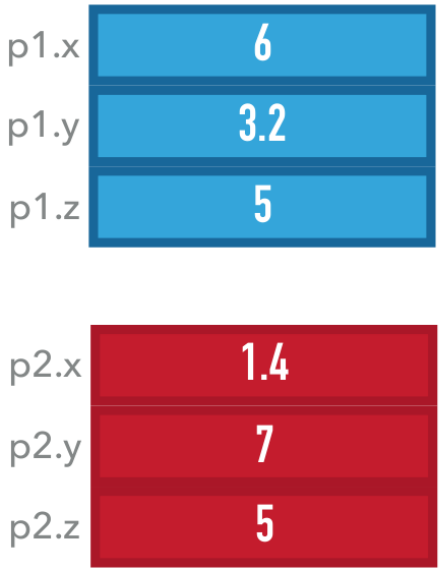




ASSEGNAMENTO E COPIA CON LE STRUTTURE

```
struct punto {
  float x;
  float y;
  float z;
};
```

```
struct punto p1;
p1.x = 1.4;
p1.y = 3.2;
p1.z = 5;
struct punto p2;
p2 = p1;
p1.x = 6;
p2.y = 7;
```



- Assegnare una struttura a un'altra struttura ne **copia tutti i campi**, es:

  x = y

  copia tutti i valori dei campi di y nei corrispondenti campi di x (ricordate che "=" non significa "*uguale a*", ma: "*assegna il valore della variabile a destra a quella a sinistra*")

- Se passate una struttura come argomento a una funzione, essa viene **copiata** (come le normali variabili, e non come gli array), quindi l'originale non viene modificato

- Non è possibile confrontare due strutture con l'operatore di uguaglianza, come avviene invece con le normali variabili non strutturate:

  x == y

  non si può fare se x e y sono due strutture (in C)

TYPEDEF

- `typedef` serve a definire il nome di un nuovo tipo di dato

- Lo possiamo usare per evitare di scrivere `struct nome_struttura` in questo modo:

```
typedef struct nome_struttura abbreviazione;
```

- Questo ci consente di usare "`abbreviazione`" al posto di `struct nome_struttura`. Esempio:

```
struct _punto2D {
  float x;
  float y;
};

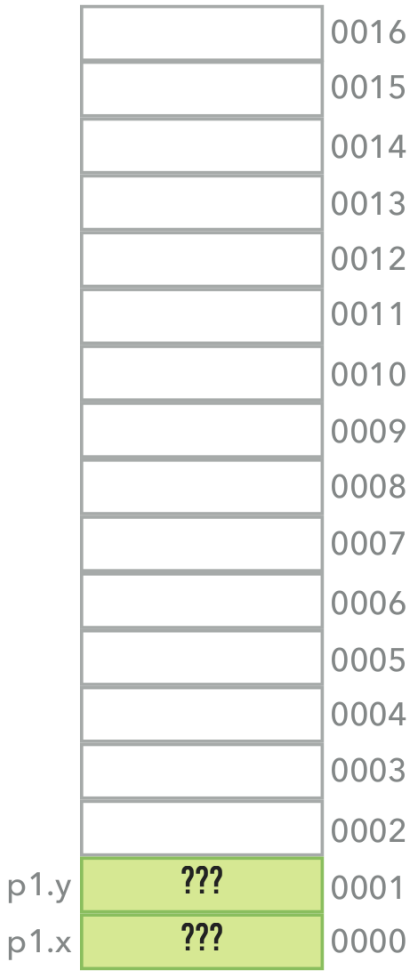
typedef struct _punto2D punto2D;
```

```
punto2D punto_medio(punto2D a, punto2D b)
{
  punto2D c;
  c.x = (a.x + b.x)/2;
  c.y = (a.y + b.y)/2;
  return c;
}

int main(int argc, char * argv[])
{
  punto2D p1;
  punto2D p2;
  punto2D m;
  p1.x = 4.0;
  p1.y = 2.0;
  p2.x = -4.0;
  p2.y = 6.0;
  m = punto_medio(p1, p2);
  exit(EXIT_SUCCESS);
}
```

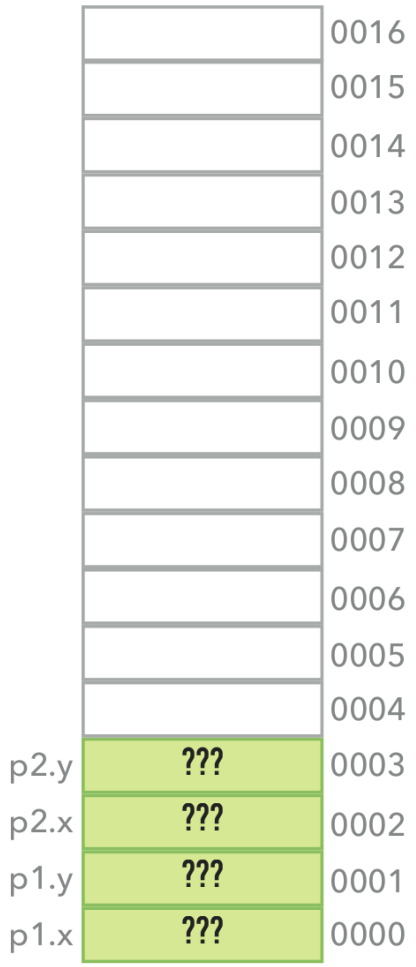
```
punto2D punto_medio(punto2D a, punto2D b)
{
  punto2D c;
  c.x = (a.x + b.x)/2;
  c.y = (a.y + b.y)/2;
  return c;
}

int main(int argc, char * argv[])
{
  punto2D p1;
  punto2D p2;
  punto2D m;
  p1.x = 4.0;
  p1.y = 2.0;
  p2.x = -4.0;
  p2.y = 6.0;
  m = punto_medio(p1, p2);
  exit(EXIT_SUCCESS);
}
```



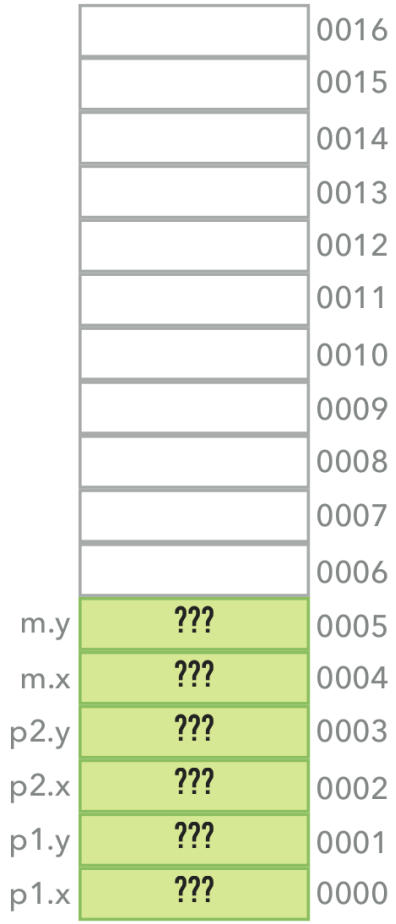
```
punto2D punto_medio(punto2D a, punto2D b)
{
  punto2D c;
  c.x = (a.x + b.x)/2;
  c.y = (a.y + b.y)/2;
  return c;
}

int main(int argc, char * argv[])
{
  punto2D p1;
  punto2D p2;
  punto2D m;
  p1.x = 4.0;
  p1.y = 2.0;
  p2.x = -4.0;
  p2.y = 6.0;
  m = punto_medio(p1, p2);
  exit(EXIT_SUCCESS);
}
```



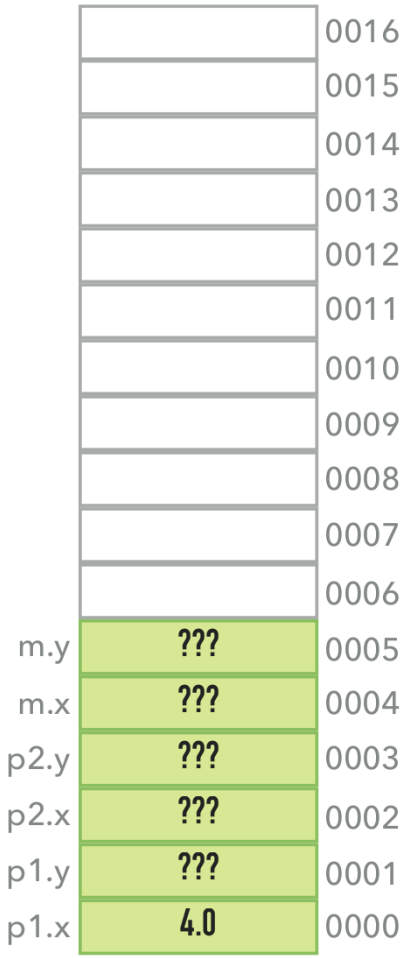
```
punto2D punto_medio(punto2D a, punto2D b)
{
  punto2D c;
  c.x = (a.x + b.x)/2;
  c.y = (a.y + b.y)/2;
  return c;
}

int main(int argc, char * argv[])
{
  punto2D p1;
  punto2D p2;
  punto2D m;
  p1.x = 4.0;
  p1.y = 2.0;
  p2.x = -4.0;
  p2.y = 6.0;
  m = punto_medio(p1, p2);
  exit(EXIT_SUCCESS);
}
```



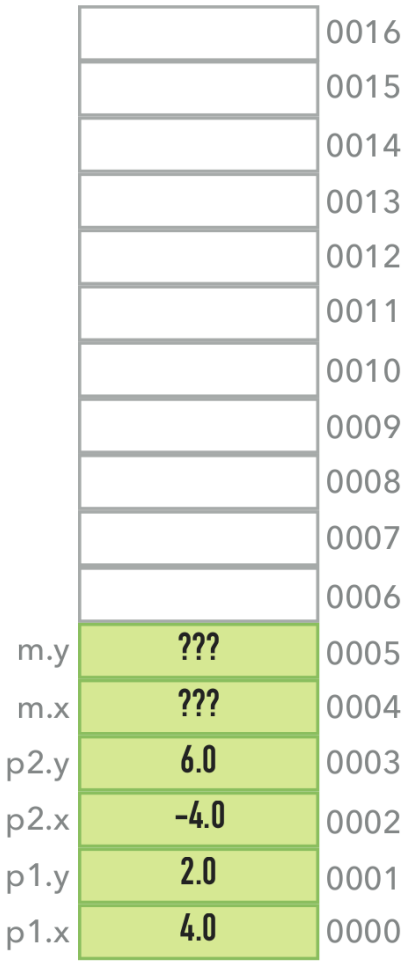
```
punto2D punto_medio(punto2D a, punto2D b)
{
  punto2D c;
  c.x = (a.x + b.x)/2;
  c.y = (a.y + b.y)/2;
  return c;
}

int main(int argc, char * argv[])
{
  punto2D p1;
  punto2D p2;
  punto2D m;
  p1.x = 4.0;
  p1.y = 2.0;
  p2.x = -4.0;
  p2.y = 6.0;
  m = punto_medio(p1, p2);
  exit(EXIT_SUCCESS);
}
```



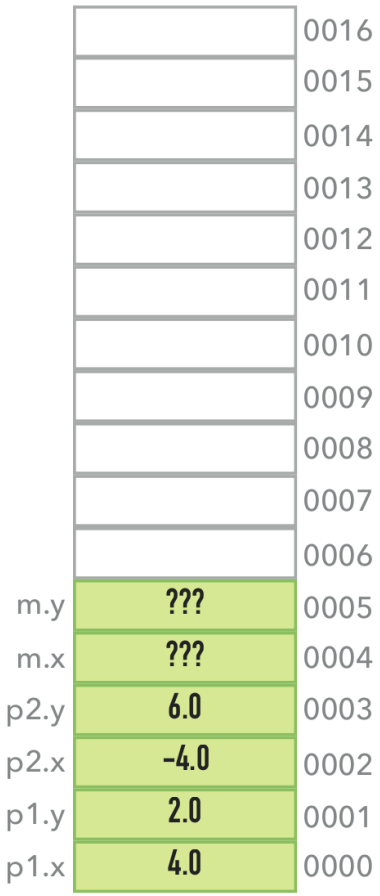
```
punto2D punto_medio(punto2D a, punto2D b)
{
  punto2D c;
  c.x = (a.x + b.x)/2;
  c.y = (a.y + b.y)/2;
  return c;
}

int main(int argc, char * argv[])
{
  punto2D p1;
  punto2D p2;
  punto2D m;
  p1.x = 4.0;
  p1.y = 2.0;
  p2.x = -4.0;
  p2.y = 6.0;
  m = punto_medio(p1, p2);
  exit(EXIT_SUCCESS);
}
```



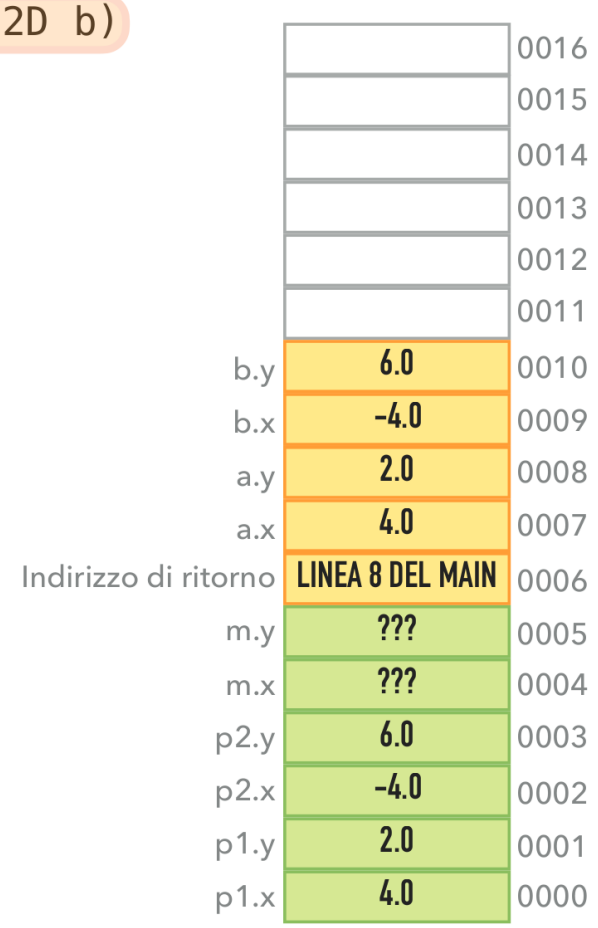
```
punto2D punto_medio(punto2D a, punto2D b)
{
  punto2D c;
  c.x = (a.x + b.x)/2;
  c.y = (a.y + b.y)/2;
  return c;
}

int main(int argc, char * argv[])
{
  punto2D p1;
  punto2D p2;
  punto2D m;
  p1.x = 4.0;
  p1.y = 2.0;
  p2.x = -4.0;
  p2.y = 6.0;
  m = punto_medio(p1, p2);
  exit(EXIT_SUCCESS);
}
```



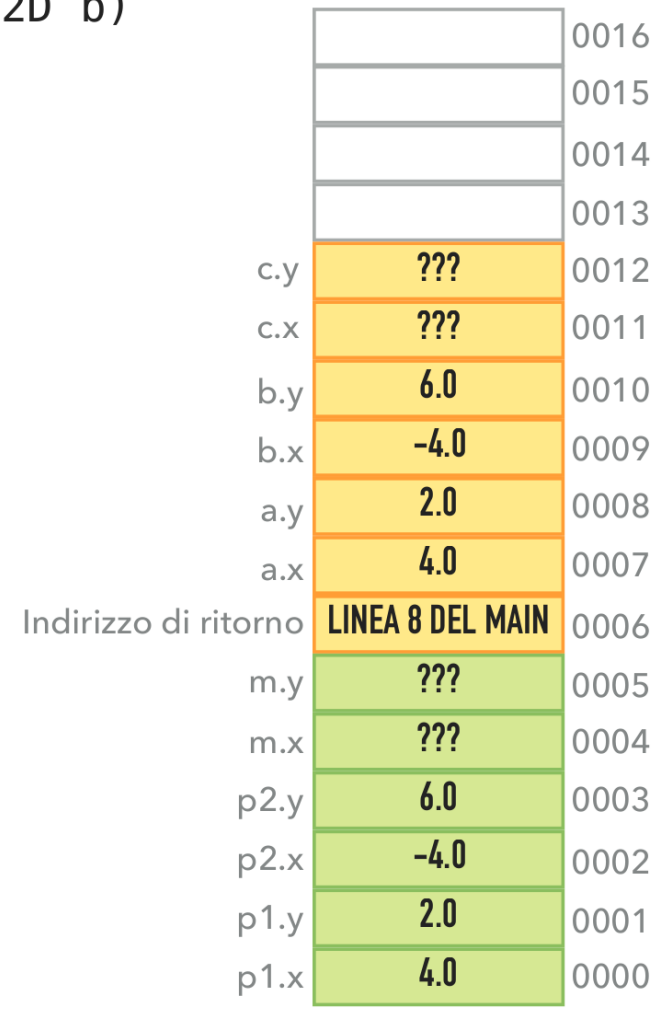
```
punto2D punto_medio(punto2D a, punto2D b)
{
  punto2D c;
  c.x = (a.x + b.x)/2;
  c.y = (a.y + b.y)/2;
  return c;
}

int main(int argc, char * argv[])
{
  punto2D p1;
  punto2D p2;
  punto2D m;
  p1.x = 4.0;
  p1.y = 2.0;
  p2.x = -4.0;
  p2.y = 6.0;
  m = punto_medio(p1, p2);
  exit(EXIT_SUCCESS);
}
```



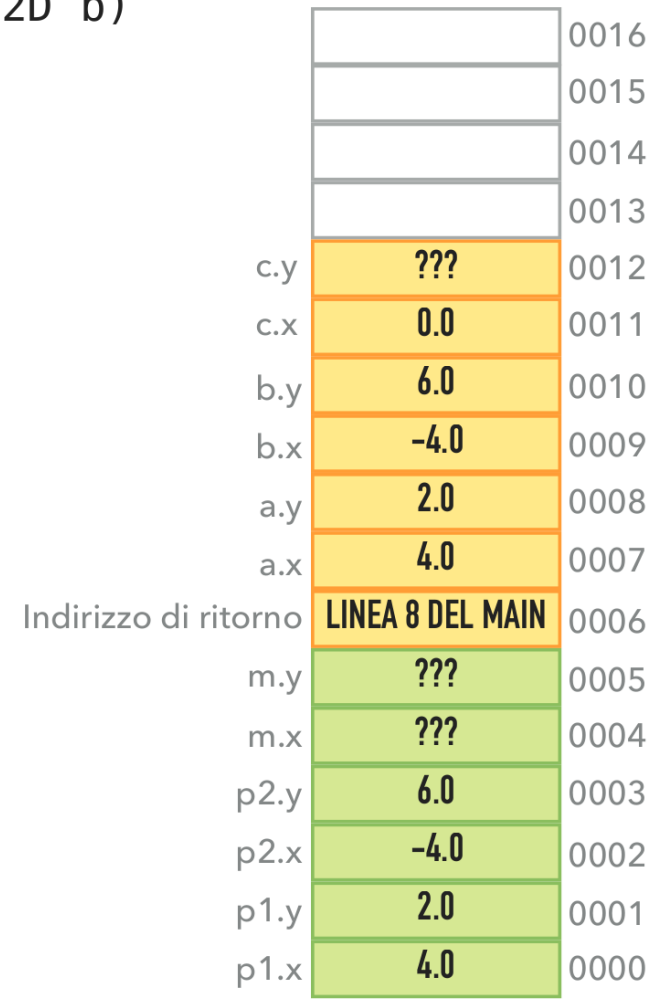
```
punto2D punto_medio(punto2D a, punto2D b)
{
  punto2D c;
  c.x = (a.x + b.x)/2;
  c.y = (a.y + b.y)/2;
  return c;
}

int main(int argc, char * argv[])
{
  punto2D p1;
  punto2D p2;
  punto2D m;
  p1.x = 4.0;
  p1.y = 2.0;
  p2.x = -4.0;
  p2.y = 6.0;
  m = punto_medio(p1, p2);
  exit(EXIT_SUCCESS);
}
```



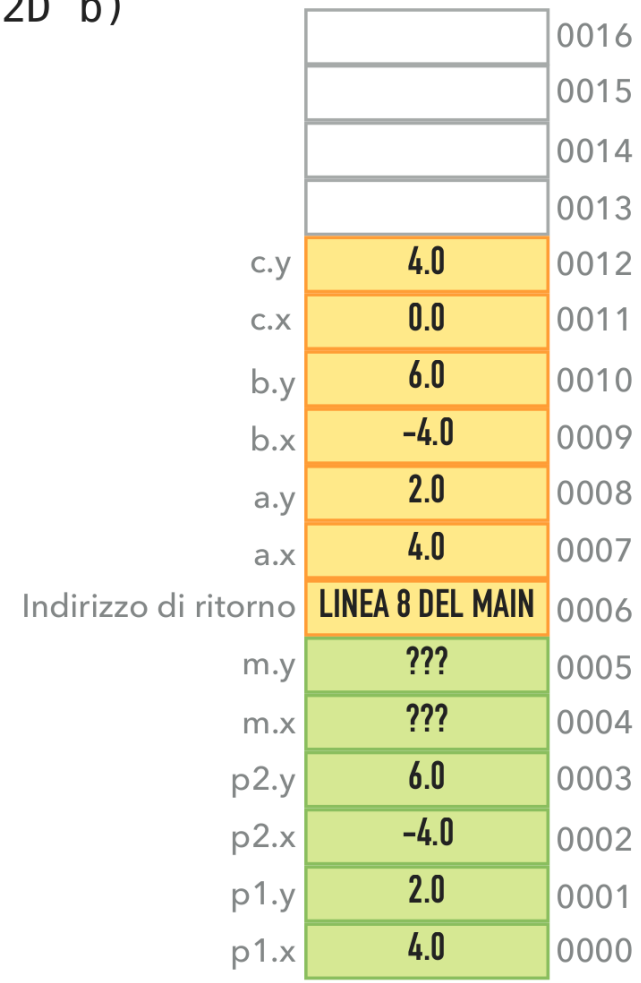
```
punto2D punto_medio(punto2D a, punto2D b)
{
  punto2D c;
  c.x = (a.x + b.x)/2;
  c.y = (a.y + b.y)/2;
  return c;
}

int main(int argc, char * argv[])
{
  punto2D p1;
  punto2D p2;
  punto2D m;
  p1.x = 4.0;
  p1.y = 2.0;
  p2.x = -4.0;
  p2.y = 6.0;
  m = punto_medio(p1, p2);
  exit(EXIT_SUCCESS);
}
```



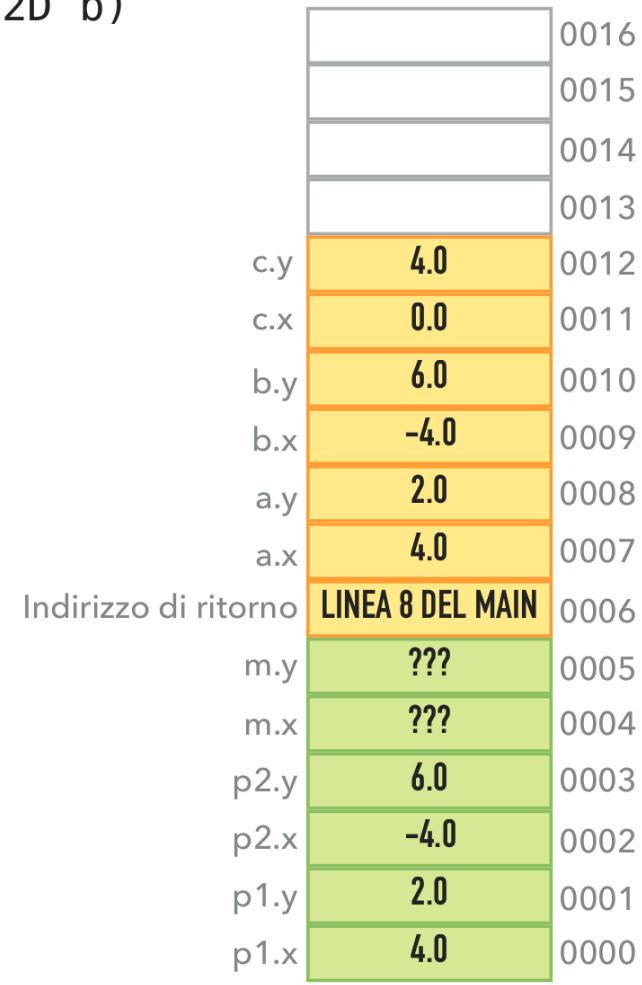
```
punto2D punto_medio(punto2D a, punto2D b)
{
  punto2D c;
  c.x = (a.x + b.x)/2;
  c.y = (a.y + b.y)/2;
  return c;
}

int main(int argc, char * argv[])
{
  punto2D p1;
  punto2D p2;
  punto2D m;
  p1.x = 4.0;
  p1.y = 2.0;
  p2.x = -4.0;
  p2.y = 6.0;
  m = punto_medio(p1, p2);
  exit(EXIT_SUCCESS);
}
```



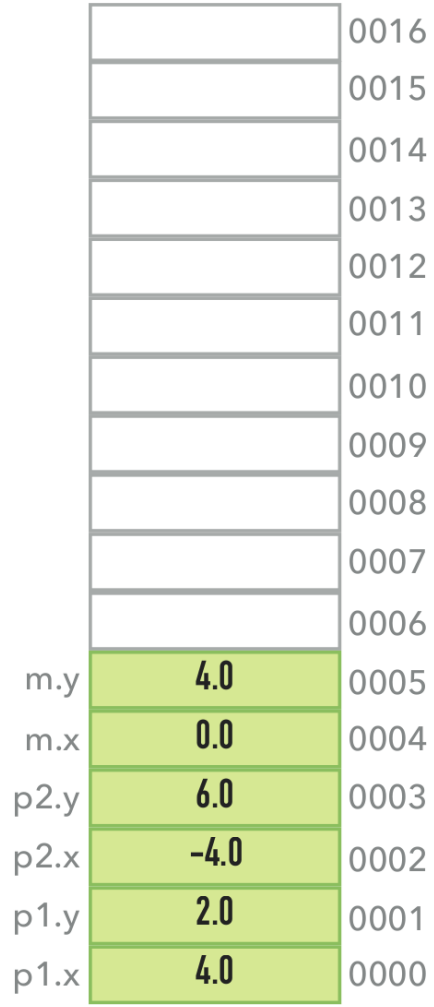
```
punto2D punto_medio(punto2D a, punto2D b)
{
  punto2D c;
  c.x = (a.x + b.x)/2;
  c.y = (a.y + b.y)/2;
  return c;
}

int main(int argc, char * argv[])
{
  punto2D p1;
  punto2D p2;
  punto2D m;
  p1.x = 4.0;
  p1.y = 2.0;
  p2.x = -4.0;
  p2.y = 6.0;
  m = punto_medio(p1, p2);
  exit(EXIT_SUCCESS);
}
```

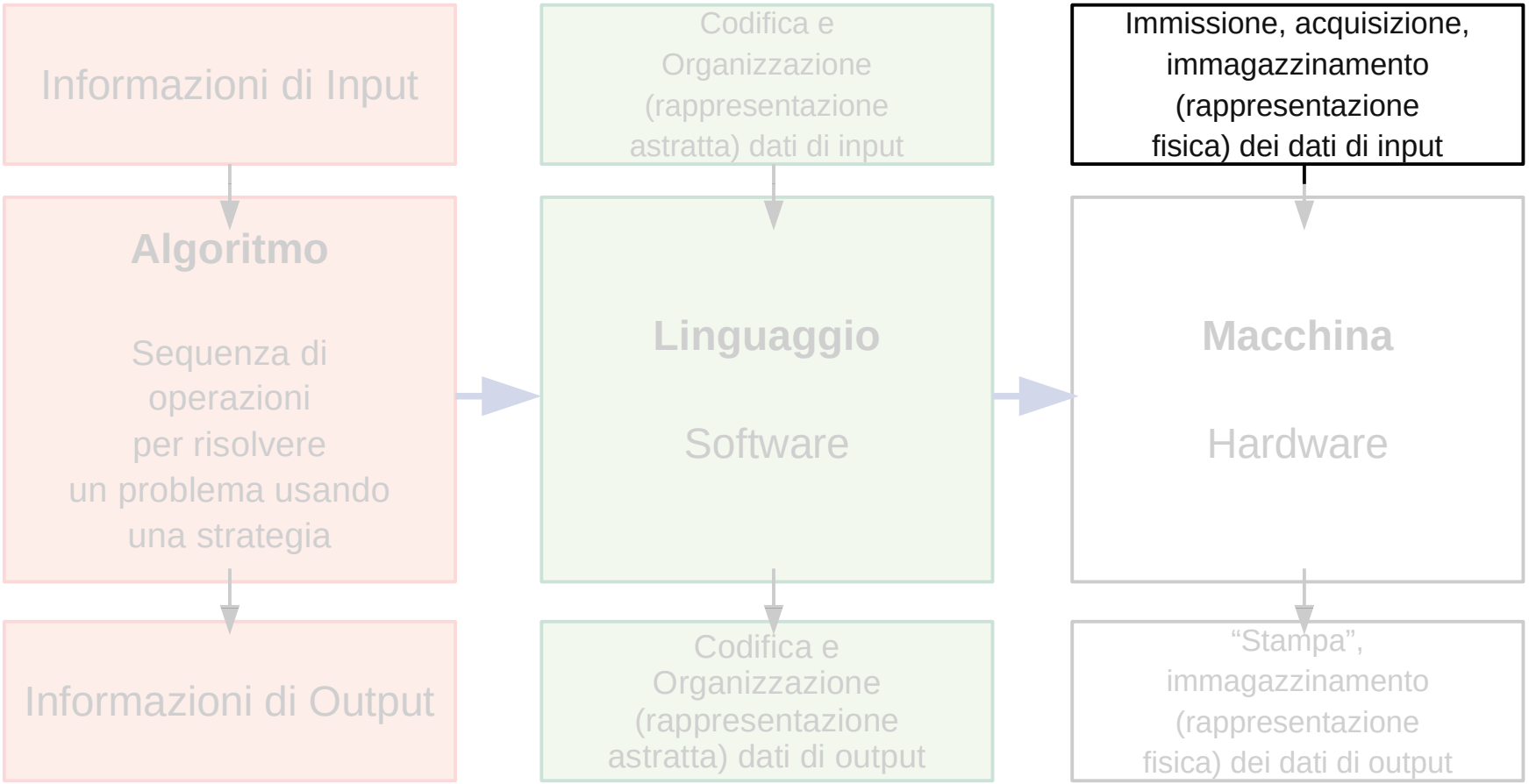


```
punto2D punto_medio(punto2D a, punto2D b)
{
  punto2D c;
  c.x = (a.x + b.x)/2;
  c.y = (a.y + b.y)/2;
  return c;
}

int main(int argc, char * argv[])
{
  punto2D p1;
  punto2D p2;
  punto2D m;
  p1.x = 4.0;
  p1.y = 2.0;
  p2.x = -4.0;
  p2.y = 6.0;
  m = punto_medio(p1, p2);
  exit(EXIT_SUCCESS);
}
```



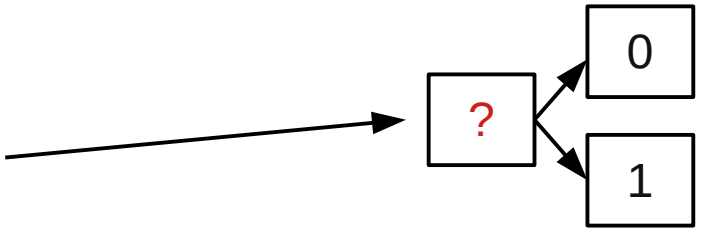
RAPPRESENTAZIONE DATI IN MEMORIA



BIT E INFORMAZIONE

Abbiamo visto durante la prima lezione che:

- Un bit è una quantità minima di informazione che può assumere solo due stati;
- I due stati sono interpretabili in diversi modi, per esempio" Vero o Falso, Bianco o Nero, Positivo o Negativo, Acceso o Spento, Sì o No, eccetera);
- Una **cifra in base 2** in informatica viene chiamata "bit" (Binary digIT). Un singolo bit come cifra binaria può valere o zero o uno;
- Una sequenza di N bit, cioè un numero binario a N cifre, può assumere 2^N diversi valori, corrispondenti a diverse combinazioni delle singole cifre binarie.



1 bit	2 bits	3 bits	4 bits	
0	00	000	0000	1000
1	01	001	0001	1001
	10	010	0010	1010
	11	011	0011	1011
		100	0100	1100
		101	0101	1101
		110	0110	1110
		111	0111	1111

[Per la rappresentazione dei numeri si vedano per esempio BMOR, Cap. 1 e BK, Cap. 1.5]

CONVERSIONE DA DECIMALE A BINARIO

- Per convertire un numero intero da base 10 a base 2 basta dividere il numero ripetutamente per 2 fino a ottenere queziente nullo
- I resti presi nell'ordine inverso forniscono le cifre del numero in base 2
- Per esempio, 140_{10} in base 2 si ottiene facendo:

	quoziente	resto
140 diviso 2 da' **70** con resto di **0**	**140**	**0**
70 diviso 2 da' **35** con resto di **0**	**70**	**0**
35 diviso 2 da' **17** con resto di **1**	**35**	**1**
17 diviso 2 da' **8** con resto di **1**	**17**	**1**
8 diviso 2 da' **4** con resto di **0**	**8**	**0**
4 diviso 2 da' **2** con resto di **0**	**4**	**0**
2 diviso 2 da' **1** con resto di **0**	**2**	**0**
1 diviso 2 da' **0** con resto di **1**	**1**	**1**
	0	

- Quindi $140_{10} = 10001100_2$

NOTA TERMINOLOGICA: KILO, MEGA, GIGA, TERA...

- Per grandi quantità di bit (b) e byte (B) si usa:
  - Kilobyte (kB) = 1024 byte (o 1000 byte...)
  - Megabyte (MB) = 1024 kB
  - Gigabyte (GB) = 1024 MB
  - Terabyte (TB) = 1024 GB

NOTAZIONE POSIZIONALE

Un numero *a* con *N* cifre a destra della virgola e *M* a sinistra della virgola espresso in base *b* si scrive come:

$$a = \sum_{i=-N}^{M-1} c_i b^i$$

Base 10:

1450.23 =

$1 \cdot 1000 + 4 \cdot 100 + 5 \cdot 10 + 0 \cdot 1 + 2 \cdot 0.1 + 3 \cdot 0.01 =$

$1 \cdot 10^3 + 4 \cdot 10^2 + 5 \cdot 10^1 + 0 \cdot 10^0 + 2 \cdot 10^{-1} + 3 \cdot 10^{-2}$

Base 2:

1010.11 =

$1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2} =$

$1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 0 \cdot 1 + 1 \cdot 1/2 + 1 \cdot 1/4 =$

10 + ¾ = 10.75

Base 16:

1A2F =

$1 \cdot 16^3 + A \cdot 16^2 + 2 \cdot 16^1 + F \cdot 16^0 =$

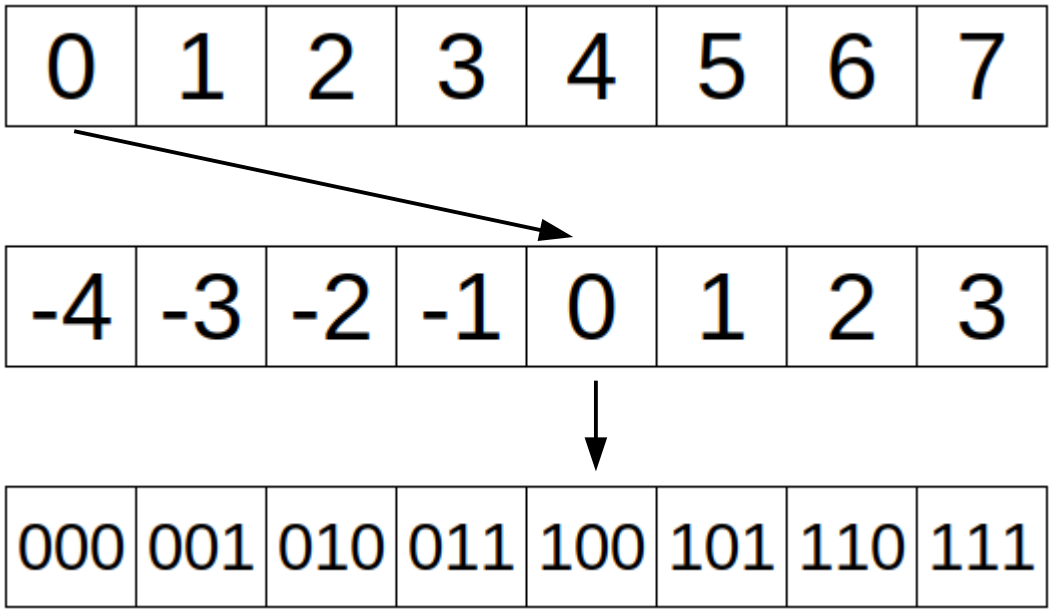
$1 \cdot 4096 + A \cdot 256 + 2 \cdot 16 + F \cdot 1 =$

$1 \cdot 4096 + 10 \cdot 256 + 2 \cdot 16 + 15 \cdot 1 = 6703$

Base 10	Binario	Esadecimale
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F
16	00010000	10
17	00010001	11
18	00010010	12
19	00010011	13
20	00010100	14
21	00010101	15
22	00010110	16
23	00010111	17
24	00011000	18
25	00011001	19
26	00011010	1A
27	00011011	1B
28	00011100	1C
29	00011101	1D
30	00011110	1E
31	00011111	1F

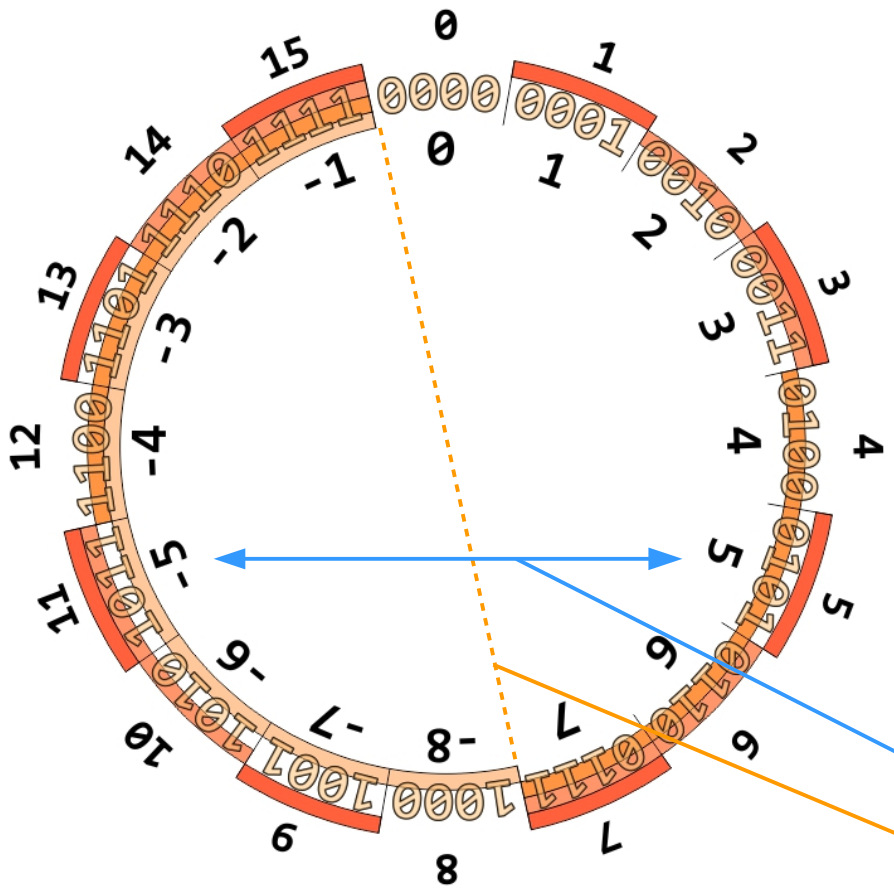
INTERI RELATIVI: RAPPRESENTAZIONE IN ECCESSO

Es.: 3 bit, $2^3 = 8$, rappresentazione in eccesso a 4 ($2^3/2 = 2^2$)



In una rappresentazione in eccesso a N, si rappresentano i numeri che vanno da **-N** fino a (2^N - 1) - N = **2^N - (N+1)**

RAPPRESENTAZIONE IN COMPLEMENTO



In generale, si dice complemento **k** di un numero *m* in base *b* rappresentato con n cifre, il numero

$m + \mathbf{k} = b^n$

Questo si può riscrivere come:

$\mathbf{k} = b^n - m = (b^n - 1) + 1 - m$

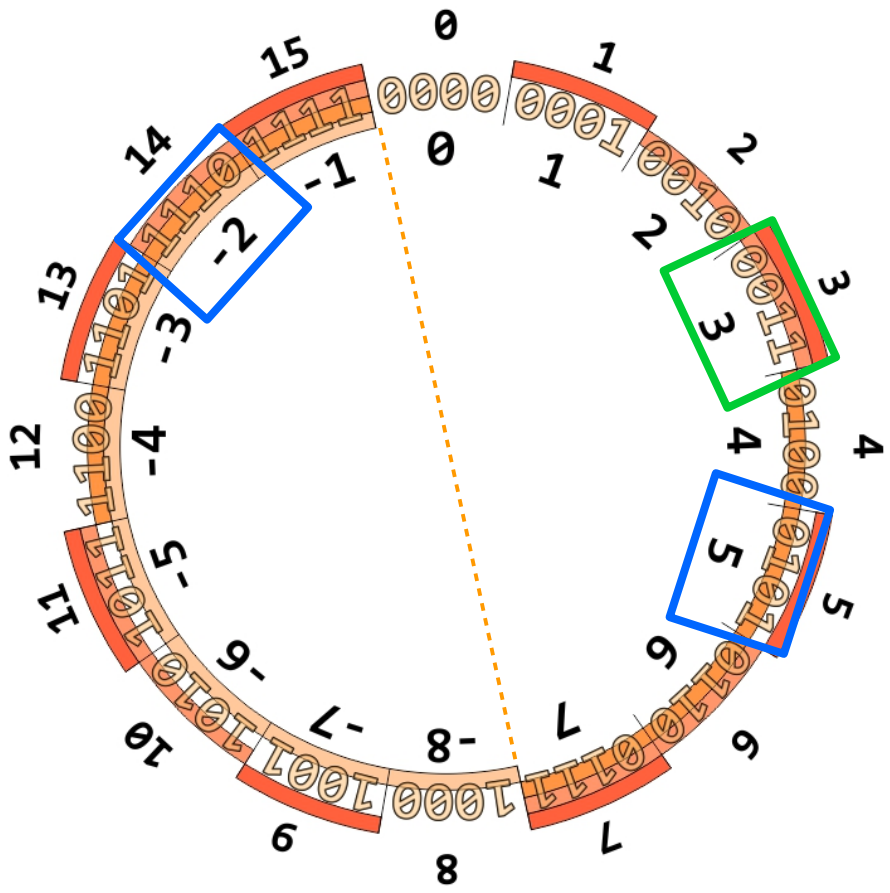
$(b^n - 1)$ è formato da una sequenza di cifre massime (es. 1111 in base 2), quindi per ottenere **k** basta trovare, dato m, **il complemento di ciascuna cifra rispetto alla cifra massima, poi sommare 1**.

Es. In base 10 (dove la cifra massima è 9), il complemento a 10 del numero 134 espresso con 4 cifre è
0134 -> 9863 + 1 = 9864

Per la base 2 il complemento di 0 è 1 e il complemento di 1 è 0
Es. per passare da 5 a -5: 0101 diventa 1010 + 1 = 1011

Si noti che tutti i numeri la cui **cifra** più significativa (quella **più a sinistra**) **è 1 sono numeri negativi**

SOMME E DIFFERENZE IN COMPLEMENTO A DUE



Proviamo a fare 5 - 2.

Possiamo scrivere 5 + (-2) usando le rispettive rappresentazioni in complemento a due con 4 cifre, ed eseguire la somma in colonna (come si fa normalmente, con il riporto, eccetera:

```
 0101 +
 1110 =
-----
10011
```

↓ Ma ho solo 4 cifre

0011

Se usiamo questa rappresentazione, **un singolo tipo di circuito può eseguire sia somme che e differenze!**

NUMERI INTERI: TIPI E DIMENSIONE

Type	Storage size	Value range
char	1 byte	-128 to 127 or 0 to 255
unsigned char	1 byte	0 to 255
signed char	1 byte	-128 to 127
int	2 or 4 bytes	-32,768 to 32,767 or -2,147,483,648 to 2,147,483,647
unsigned int	2 or 4 bytes	0 to 65,535 or 0 to 4,294,967,295
short	2 bytes	-32,768 to 32,767
unsigned short	2 bytes	0 to 65,535
long	8 bytes or (4bytes for 32 bit OS)	-9223372036854775808 to 9223372036854775807
unsigned long	8 bytes	0 to 18446744073709551615

Nota: In C utilizzare la funzione sizeof(type) per avere l'effettiva dimensione in byte, che può variare a seconda degli standard

I numeri reali non sono rappresentati esattamente all'interno di un computer, essendo il loro insieme denso mentre le possibilità di un calcolatore sono solo di immagazzinare informazioni discrete

Tuttavia esistono metodi e convenzioni per rappresentarli in modo approssimato, tenendo eventualmente conto delle differenze tra la rappresentazione e il numero vero

Si noti che non si tratta solo di un problema di numero di cifre (per esempio il π), ma come vedremo anche per esempio il numero 0.1 (in base 10) non può essere rappresentato con un numero finito di cifre in base 2)

RAPPRESENTAZIONE NORMALE IN VIRGOLA MOBILE

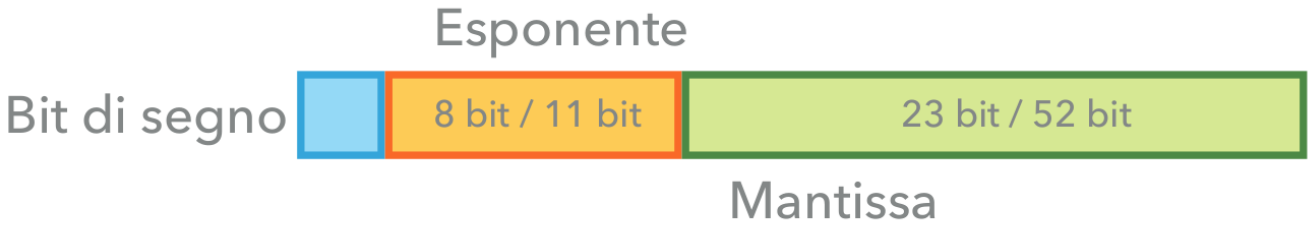
Per i numeri reali si adotta una convenzione (definita nello standard IEEE 754) che definisce come sono rappresentati i numeri floating point (in virgola mobile) e come funzionano le operazioni su di essi.

Il numero totale di bit può cambiare a seconda della precisione desiderata:
single precision: 32 bit, double precision: 64 bit

I bit a disposizione vengono utilizzati per tre informazioni diverse: segno, esponente e mantissa

Nota: Vengono assegnate delle rappresentazioni per risultati impossibili da calcolare dette NaN (not a number), e per risultati “infiniti” +Inf e -Inf

RAPPRESENTAZIONE IN FORMA NORMALE (O CANONICA)



$$\pm \text{mantissa} \times 2^{\text{esponente}}$$

- Si converte il numero con la virgola da base 10 in base 2 (si convertono separatemente parte intera e parte decimale e poi si uniscono)
- Si scrive il numero in base 2 spostando la virgola in modo che sia 1 la parte intera, in modo da avere solo una cifra unitaria alla sua sinistra, e scegliendo di conseguenza l'esponente da dare a 2 (si noti che ogni spostamento della virgola corrisponde a moltiplicare o dividere per 2, analogamente alla base 10)
- Si assegna un bit per il segno (0 significa positivo, 1 significa negativo)
- Si usano 8 bit (o 11 bit) per rappresentare l'esponente in eccesso a 127
- Per la mantissa, si usano 23 bit (o 53 bit). Viene immagazzinata allineandola a sinistra omettendo il primo 1
- Se l'esponente è composto di soli 0, allora non si sottintende l'omissione dell'1 iniziale (forma denormalizzata)

CONVERSIONE PARTE FRAZIONARIA

Per convertire un numero decimale con parte intera e parte frazionaria, si converte la parte intera come sappiamo. Dopo la virgola scriveremo la conversione della parte frazionaria. Per convertire la parte frazionaria:

Moltiplichiamo la parte frazionaria del numero dato per 2;

Continuiamo a moltiplicare il risultato ottenuto per 2 tenendo presente che, se il numero ottenuto è maggiore di 1 sottraiamo 1 prima della successiva moltiplicazione per 2;

Andiamo avanti fino a quando non otteniamo o un risultato uguale a uno, oppure fino a quando otteniamo un risultato già ottenuto in precedenza;

Quando otteniamo un risultato già ottenuto in precedenza significa che il numero binario è periodico. Ciò può accadere anche se il numero decimale di partenza non era periodico.

Il numero binario da noi cercato è dato dalle **parti intere** dei **prodotti** ottenuti, ordinati partendo dal primo risultato.

CONVERSIONE DECIMALE IN FLOATING POINT

Per convertire 17.15_{10} nel formato floating point:

Convertire 17 in binario: $17_{10} = 10001_2$

Convertire 0.15 in binario 0.00(1001)

$17.15_{10} = 10001.00100110011001..._2$

Spostare la visrgola di 4 posti (quindi esponente = 4) per ottenere M = m - 1 = 00010010011001...

Per l'esponente: $E = e + 127 = 131_{10} = 1000\ 0011_2$

Il numero è positivo, quindi S=0



TIPI FLOAT, DIMENSIONE E INTERVALLI

Type	Storage size	Value range	Precision
float	4 byte	1.2E-38 to 3.4E+38	6 decimal places
double	8 byte	2.3E-308 to 1.7E+308	15 decimal places
long double	10 byte	3.4E-4932 to 1.1E+4932	19 decimal places

INSIEME DI TUTTI E SOLI I NUMERI REALI RAPPRESENTABILI

Data una base β intera, maggiore o uguale a 2, un numero *t* di cifre significative, un intervallo [L, U] di esponenti *e* interi, allora l'insieme dei numeri rappresentabili in virgola mobile in forma normale è:

$$\mathbb{F}(\beta, t, L, U) = \{0\} \cup \left\{ x \in \mathbb{R} : x = (-1)^s \beta^e \sum_{i=1}^{t} d_i \beta^{-i} \right\}$$

Nota: in questa definizione non consideriamo la forma denormalizzata

con le condizioni:

$\beta \in \mathbb{N}, \beta \geq 2$

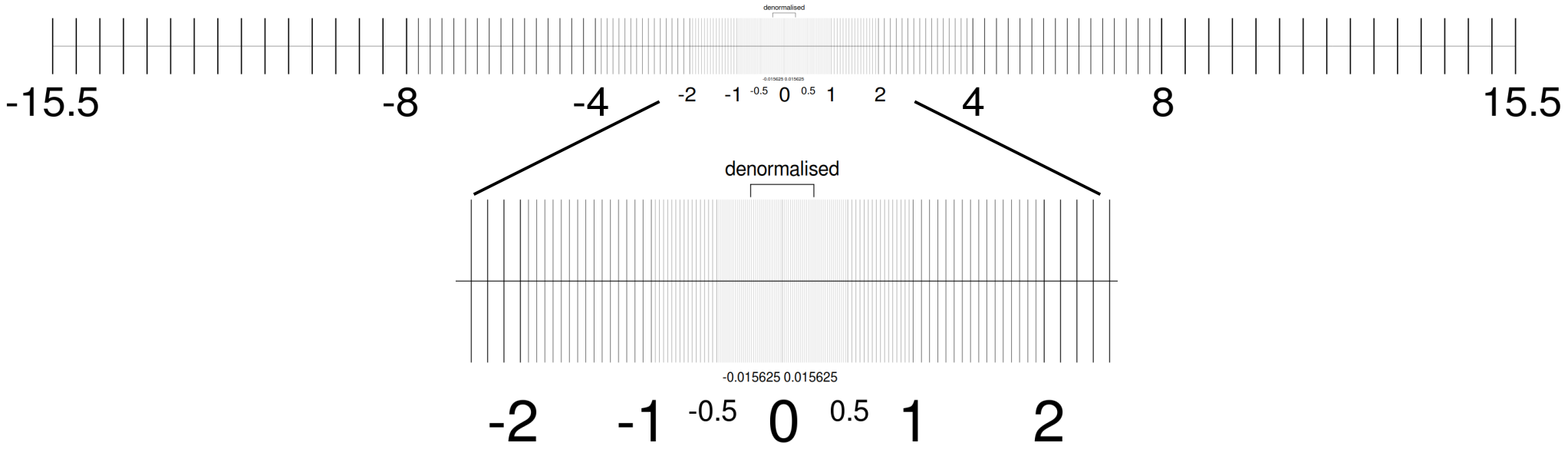
$0 \leq d_i \leq \beta - 1$

$d_1 \neq 0$

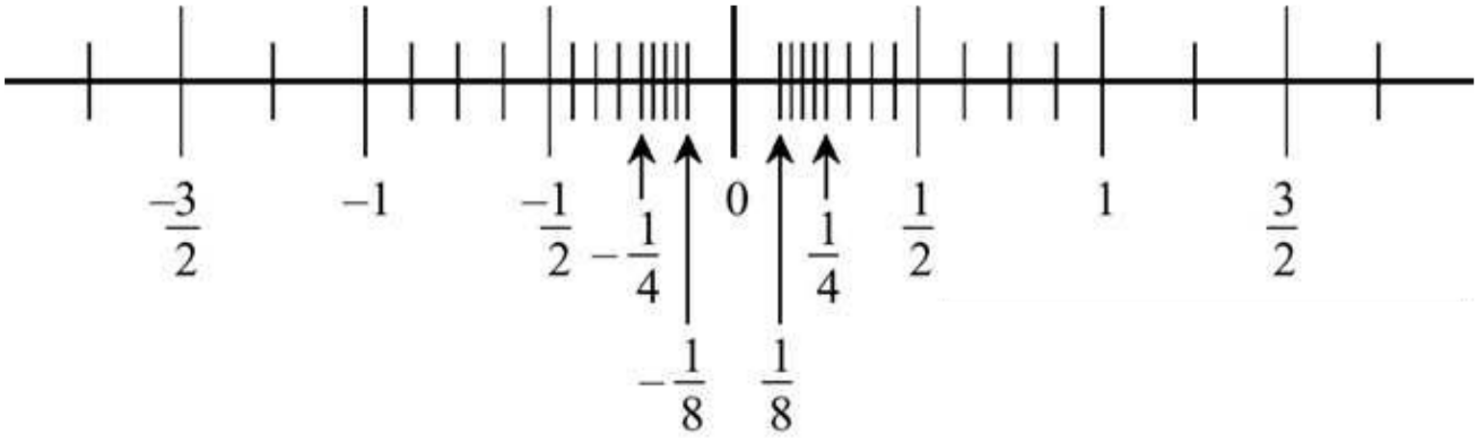
$L \leq e \leq U$ (Normalmente L è negativo e U è positivo, per avere esponenti sia positivi che negativi)

DENSITÀ VARIABILE DEI FLOATING POINT

Tutti e soli i numeri floating point rappresentabili in forma normale, avendo a disposizione solo un byte (8 bit), usando 4 bit per la mantissa e 3 bit per l'esponente

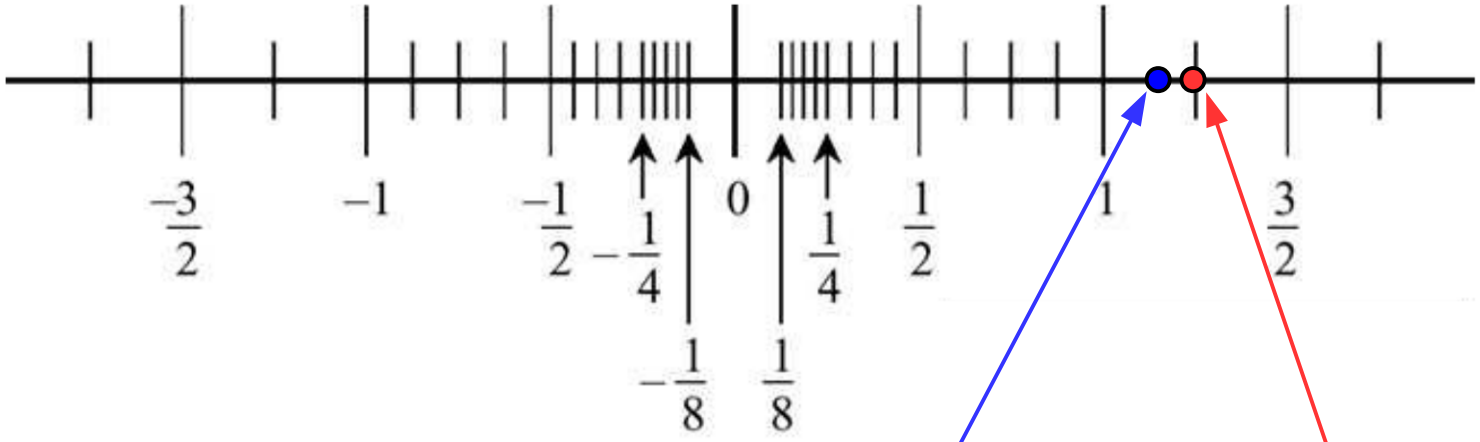


DISTRIBUZIONE SULL'ASSE REALE DEI FLOATING POINT



- numeri che capitano in mezzo alle divisioni saranno rappresentati solo in modo approssimato
- frazioni che non sono potenze di 2 (per esempio il numero 0.1_{10}) non sono rappresentabili esattamente
- la densità di divisioni non è costante, i numeri si addensano attorno allo 0 e si diradano per valori assoluti via via maggiori

ERRORI DI APPROSSIMAZIONE ASSOLUTI E RELATIVI



Sia x un numero reale e $\hat{x}$ una sua approssimazione

Errore assoluto: $E_a = |x - \hat{x}|$

Errore relativo $E_r = \dfrac{|x - \hat{x}|}{|x|}$ (se $x \neq 0$)

EPSILON DI MACCHINA ED ERRORE RELATIVO

Machine precision (o *machine epsilon*)

$B =$ base $\quad t =$ cifre significative (mantissa)

$$E_{\text{mach}} = B^{1-t}$$

oppure

$$E_{\text{mach}} = \frac{1}{2} B^{1-t}$$

È un indice del massimo errore relativo nell'approssimazione di un numero diverso da 0

$$\left| \frac{\hat{x} - x}{x} \right| \leq E_{\text{mach}}$$

Per i float a 32 bit (senza contare la forma denormalizzata):

In base 2: $L = -127$, $U = 128$, $E_{\text{mach}} = 2^{-23}$

In base 10: $L \simeq -38$, $U = 38$, $E_{\text{mach}} \simeq 10^{-7}$

Per i double a 64 bit (senza contare la forma denormalizzata):

In base 2: $L = -1023$, $U = 1024$, $E_{\text{mach}} = 2^{-53}$

In base 10: $L \simeq -308$, $U = 308$, $E_{\text{mach}} \simeq 10^{-16}$

In una rappresentazione a virgola mobile, i numeri rappresentabili esattamenti sono più 'densi' intorno allo 0, e si diradano a mano a mano che aumenta il valore assoluto. In questo modo l'errore relativo è limitato, indipendentemente dalla grandezza del numero

IL PROBLEMA DELLE APPROSSIMAZIONI

A causa della non esattezza delle rappresentazioni di numeri reali non è detto che le operazioni siano associative o commutative

Operazioni matematicamente equivalenti possono dare risultati diversi a causa delle diverse approssimazioni

A volte è consigliabile controllare non l'uguaglianza ma guardate se si è sufficientemente vicini al valore corretto

In una sequenza di operazioni su numeri in virgola mobile, gli errori possono amplificarsi

La disciplina che si si occupa dei problemi legati ai calcoli approssimati si chiama Analisi Numerica

RAPPRESENTAZIONE DI CARATTERI

Ogni carattere è rappresentato da un numero intero non negativo

Secondo la convenzione ASCII, un carattere occupa 8 bit (un byte)

Esempio: ’A’ = 65, ’B’ = 66, . . . , ’Z’ = 90, ’a’ = 97, ’b’ = 98

Le cifre non corrispondono ai valori numerici, es.: ’0’ = 48, ’1’ = 49

Ci sono caratteri speciali come il ritorno a capo (line feed, '\n') = 10

Codice Unicode: caratteri ‘larghi’ (due byte) per rappresentare caratteri di tutti i vari alfabeti

ASCII control characters

00	NULL	(Null character)
01	SOH	(Start of Header)
02	STX	(Start of Text)
03	ETX	(End of Text)
04	EOT	(End of Trans.)
05	ENQ	(Enquiry)
06	ACK	(Acknowledgement)
07	BEL	(Bell)
08	BS	(Backspace)
09	HT	(Horizontal Tab)
10	LF	(Line feed)
11	VT	(Vertical Tab)
12	FF	(Form feed)
13	CR	(Carriage return)
14	SO	(Shift Out)
15	SI	(Shift In)
16	DLE	(Data link escape)
17	DC1	(Device control 1)
18	DC2	(Device control 2)
19	DC3	(Device control 3)
20	DC4	(Device control 4)
21	NAK	(Negative acknowl.)
22	SYN	(Synchronous idle)
23	ETB	(End of trans. block)
24	CAN	(Cancel)
25	EM	(End of medium)
26	SUB	(Substitute)
27	ESC	(Escape)
28	FS	(File separator)
29	GS	(Group separator)
30	RS	(Record separator)
31	US	(Unit separator)
127	DEL	(Delete)

ASCII printable characters

32	space	64	@	96	`
33	!	65	A	97	a
34	"	66	B	98	b
35	#	67	C	99	c
36	$	68	D	100	d
37	%	69	E	101	e
38	&	70	F	102	f
39	'	71	G	103	g
40	(	72	H	104	h
41	)	73	I	105	i
42	*	74	J	106	j
43	+	75	K	107	k
44	,	76	L	108	l
45	-	77	M	109	m
46	.	78	N	110	n
47	/	79	O	111	o
48	0	80	P	112	p
49	1	81	Q	113	q
50	2	82	R	114	r
51	3	83	S	115	s
52	4	84	T	116	t
53	5	85	U	117	u
54	6	86	V	118	v
55	7	87	W	119	w
56	8	88	X	120	x
57	9	89	Y	121	y
58	:	90	Z	122	z
59	;	91	[	123	{
60	<	92	\	124	|
61	=	93	]	125	}
62	>	94	^	126	~
63	?	95	_		

Extended ASCII characters

128	Ç	160	á	192	└	224	Ó
129	ü	161	í	193	┴	225	ß
130	é	162	ó	194	┬	226	Ô
131	â	163	ú	195	├	227	Ò
132	ä	164	ñ	196	─	228	õ
133	à	165	Ñ	197	┼	229	Õ
134	å	166	ª	198	ã	230	µ
135	ç	167	º	199	Ã	231	þ
136	ê	168	¿	200	╚	232	Þ
137	ë	169	®	201	╔	233	Ú
138	è	170	¬	202	╩	234	Û
139	ï	171	½	203	╦	235	Ù
140	î	172	¼	204	╠	236	ý
141	ì	173	¡	205	═	237	Ý
142	Ä	174	«	206	╬	238	¯
143	Å	175	»	207	¤	239	´
144	É	176	▒	208	ð	240	≡
145	æ	177	▓	209	Ð	241	±
146	Æ	178	│	210	Ê	242	‗
147	ô	179	┤	211	Ë	243	¾
148	ö	180	┤	212	È	244	¶
149	ò	181	Á	213	ı	245	§
150	û	182	Â	214	Í	246	÷
151	ù	183	À	215	Î	247	¸
152	ÿ	184	©	216	Ï	248	°
153	Ö	185	╣	217	┘	249	¨
154	Ü	186	║	218	┌	250	·
155	ø	187	╗	219	█	251	¹
156	£	188	╝	220	▄	252	³
157	Ø	189	¢	221	¦	253	²
158	×	190	¥	222	Ì	254	■
159	ƒ	191	┐	223	▀	255	nbsp